

Personalized AI Trim Engine

The ChatGPT of video trimming

Intent-driven editing on Gemini 2.5, DeepAgents, LangGraph, and Video-Editor MCP

Problem

Today's trim tools are rule-based: silence removal, filler-word removal, chapter splits. They cannot follow requests like 'make it feel cinematic', 'remove awkward moments', 'keep only questions', or 'trim to a 30-second Reels cut'. Users end up back in Premiere.

Proposed Solution

A flexible intent-based engine that turns any natural-language edit request into a verifiable Edit Decision List, then executes it via the Video-Editor MCP.

Gemini 2.5 handles perception & language. DeepAgents (LangChain) provides planning, sub-agent delegation, virtual filesystem, and durable execution on the LangGraph runtime.

Deliverables in this document

1. Product goals, personas, and success metrics
2. System architecture (5 diagrams) and prompt-to-edit pipeline
3. Core data contracts (EditIntent, TimelineIndex, EDL) with JSON schemas
4. Component design for each of the four specialist agents
5. Working-prototype implementation plan with pseudocode
6. Edge cases and failure handling
7. Sample outputs for 22 different prompts
8. Evaluation strategy and 12-week rollout roadmap

1. Executive Summary

Video trimming today is a mechanical operation: users tell software *which seconds* to cut. The Personalized AI Trim Engine inverts that relationship — users tell the system *what they want the video to feel like*, and the system figures out which seconds to cut. Everything from “remove all my ums” to “make it feel like an A24 trailer” is expressed as intent, resolved against a rich perceptual index of the footage, and rendered deterministically.

The engine is built on four battle-tested pieces: **Gemini 2.5** for multimodal video understanding and language, **DeepAgents** (LangChain’s agent harness) for planning, sub-agent delegation, and the virtual filesystem that keeps intermediate artifacts auditable, **LangGraph** for the durable, state-machine runtime, and the **Video-Editor MCP** server (Video Jungle) for the actual video operations — upload, semantic search, edit generation, DaVinci hand-off.

The goal, per the assignment, is **not** to build a fixed list of features. It is to build a schema — **EditIntent** — general enough that unseen prompts map onto it with zero code changes, and a planner smart enough to always produce *the closest possible edit*.

Key design decisions at a glance

Language & perception	Gemini 2.5 Pro for video (1 fps native sampling, MM:SS timestamp addressing, up to 1 hour context); Gemini 2.5 Flash for structured intent parsing.
Agent runtime	DeepAgents (LangChain) on the LangGraph runtime — planning via write_todos, sub-agents for isolated work, virtual filesystem for artifacts, HITL checkpoints.
Video operations	Video-Editor MCP (burningion/video-editing-mcp) — add-video, search-videos, generate-edit-from-single-video, update-video-edit, edit-locally (OTIO → DaVinci).
Contract between agents	Three typed intermediates: EditIntent (what the user wants), TimelineIndex (what is in the video), EDL (what to cut). All JSON-schema validated, all inspectable.
Generalization strategy	Intent taxonomy has six families (cleanup, scene, person/object, emotion/action, speech/content, cinematic/story). New prompts are re-projected onto the same axes rather than triggering new code paths.
Failure handling	Explicit conditional edges (LOW_CONFIDENCE, NO_MATCH, OVER_CUT, UNDER_CUT), a critic loop capped at three refinements, HITL fallback.

2. Goals, Personas & Success Metrics

2.1 Product goals

- **Talk-to-an-editor UX.** A single prompt box, no timelines, no waveforms unless the user asks.
- **Generalization first.** Any of the six intent families works for any video; unseen prompts still land on a reasonable EDL.
- **Verifiable output.** Every cut has a JSON reason attached: the user can inspect, override, or roll back.
- **Deterministic execution.** LLMs propose; ffmpeg / MCP disposes. The render step is pure and reproducible from the EDL.
- **Editor hand-off.** Never a dead end — export OTIO into DaVinci Resolve for last-mile polishing.

2.2 Personas

Content creator (primary)	Solo YouTuber / Reels creator recording 20–60 min of raw takes. Wants a 30–90 s cut in one shot, with the flexibility to say “make it snappier” and iterate.
Podcaster / interviewer	Two-camera, long-form conversations. Needs speaker-aware cutting, question/answer isolation, and topic-based trims (“keep only the pricing discussion”).
Marketer	Needs product-focused cuts, sponsor removal, brand-safe pacing, and multi-format exports (Reels 9:16, YouTube 16:9, Square 1:1) from one source.
Prosumer editor	Uses the engine as a first-pass rough cut, then continues in DaVinci Resolve via the OTIO export.

2.3 Success metrics

- **Intent match rate.** % of prompts where the produced EDL satisfies the intent per human review. Target: 85% at v1.
- **Zero-shot generalization rate.** % of prompts *never seen in eval set* that produce an acceptable edit. Target: 70%.
- **Time to first cut.** Median wall-clock from prompt to preview MP4 for a 10 min source. Target: < 90 s.
- **Iteration cost.** Median tokens + MCP calls to satisfy a follow-up (“shorter”, “more energy”). Target: < 20% of first-cut cost.
- **Over-cut / under-cut rate.** % of edits where duration is off target by > 20%. Target: < 10%.

3. System Architecture

The system is a four-agent LangGraph state machine wrapped in the DeepAgents harness. Each agent owns a narrow responsibility; typed artifacts flow between them via the DeepAgents virtual filesystem. Gemini is the reasoning + perception engine; the Video-Editor MCP is the effector.

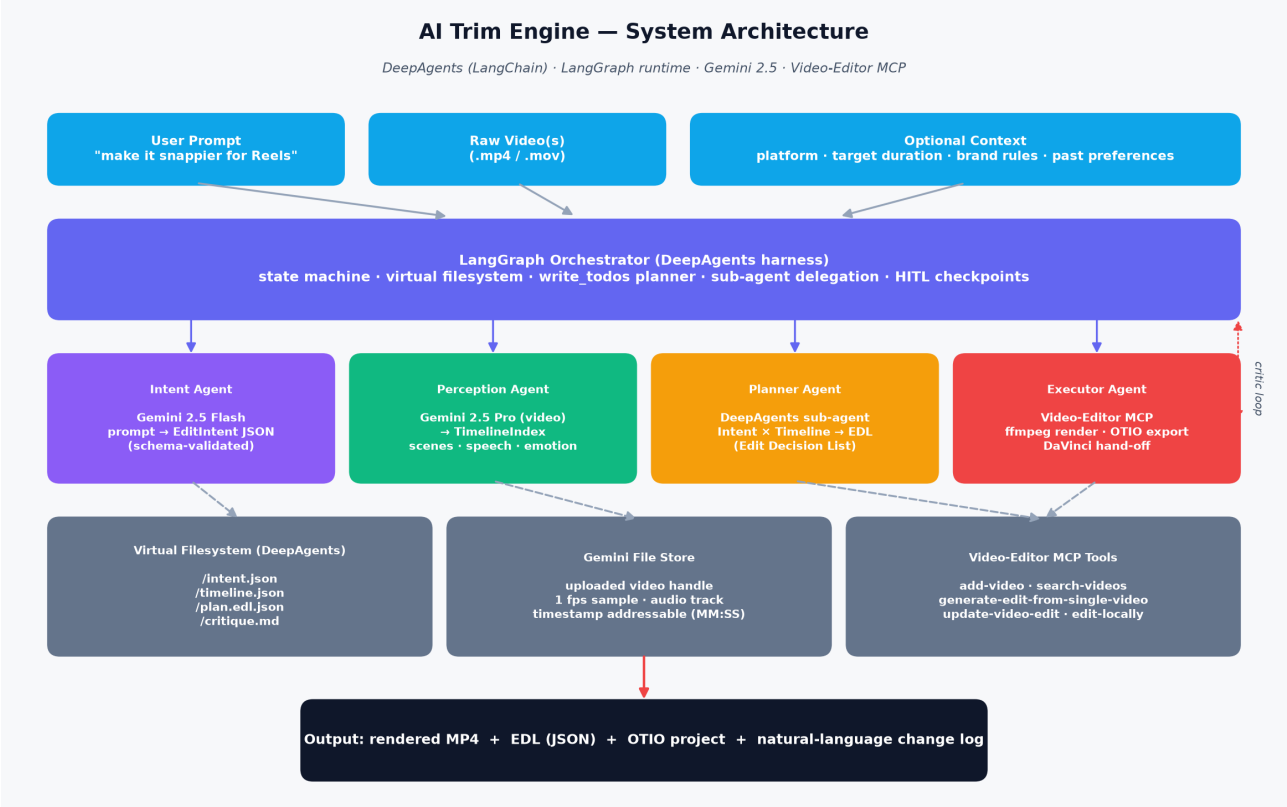


Figure 1. High-level architecture. User inputs feed a LangGraph orchestrator that dispatches four specialist agents. Artifacts are persisted in DeepAgents' virtual filesystem and rendered via the Video-Editor MCP.

3.1 Why this stack?

Gemini 2.5	Native multimodal on video & audio; supports MM:SS timestamp addressing; up to 1 h at default resolution and 3 h at low resolution; strong at “identifying specific moments” per Google’s own 2.5 benchmarks. This removes the need for a separate ASR + scene-detection + object-detection stack.
DeepAgents	Batteries-included harness: write_todos planner, sub-agent delegation, virtual filesystem, MCP tool integration, HITL. Removes ~1 kLOC of glue we would otherwise write.
LangGraph	Durable execution, streaming, replay, and typed state transitions — critical for a long-running pipeline where a Gemini call may take 30–60 s and we need retries without redoing work.
Video-Editor MCP	Ships the exact primitives we need — add-video, search-videos, generate-edit-from-single-video, update-video-edit, edit-locally (OTIO export to DaVinci Resolve). MCP is the interoperability layer.

3.2 Component responsibilities

Intent Agent	Parses the user prompt (+ context: platform, duration, brand rules, prior turns) into a schema-validated EditIntent. Uses Gemini 2.5 Flash in JSON mode. Emits an ambiguity_score; anything above 0.6 triggers a clarifying question via HITL.
Perception Agent	Runs Gemini 2.5 Pro over the uploaded video and produces the TimelineIndex: per-second scene labels, transcripts, filler spans, silence spans, emotions, objects, actions, shot type, hook / interest scores. Cached by video hash; regenerated only on new footage.
Planner Agent	The heart of the system. Consumes EditIntent and TimelineIndex, runs semantic retrieval via the MCP search-videos tool for content-based filters, then synthesizes a concrete EDL. Applies platform-specific rules (Reels < 90 s, TikTok 9:16, hook in the first 3 s).
Executor Agent	Turns the EDL into pixels. Calls MCP generate-edit-from-single-video (or multi-video for stitched sources), then writes back an update-video-edit for any post-render fixups. Optionally hands off via edit-locally for DaVinci.

4. Prompt → Edit Pipeline

The pipeline is a ten-node LangGraph. Nodes are pure functions over a shared GraphState TypedDict; edges are either unconditional, conditional (the critic loop), or HITL (final confirm).

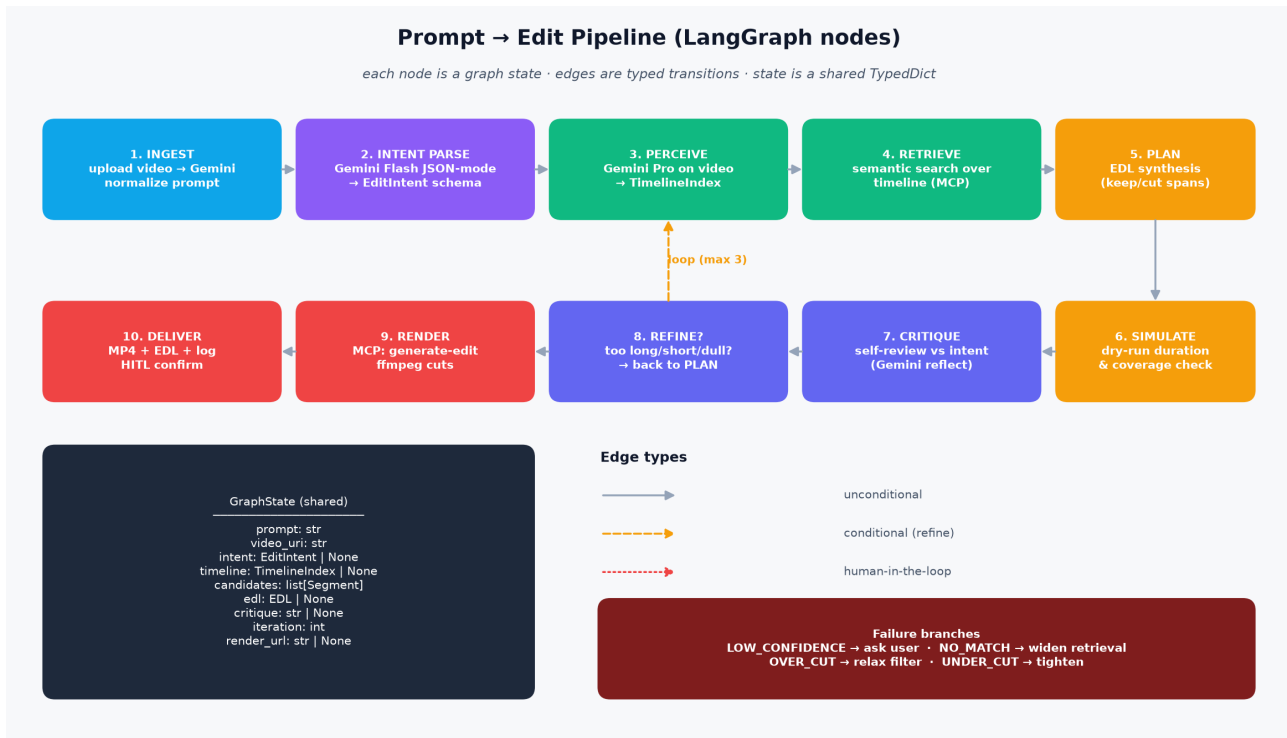


Figure 2. The ten LangGraph nodes. The critic loop between nodes 8 and 5 is bounded at three iterations. Failure branches trigger widening (retrieval), relaxing (filters), or asking the user (low confidence).

4.1 Node-by-node

1. INGEST	Upload video to Gemini via the Files API (returns a file handle); normalize the prompt (strip pleasantries, resolve pronouns from recent conversation).
2. INTENT PARSE	Gemini 2.5 Flash with JSON-mode + schema. Emits EditIntent + ambiguity_score.
3. PERCEIVE	Gemini 2.5 Pro with the video handle. Prompt asks for the full TimelineIndex. Cached by SHA256(video).
4. RETRIEVE	For content-heavy intents (“keep only pricing”), issue MCP search-videos queries derived from the intent. Merges embeddings-based hits with the perceived index.
5. PLAN	Compose the EDL: apply KEEP / REMOVE masks to candidate spans, respect duration bounds, insert transitions, order for narrative arc if style requests it.
6. SIMULATE	Compute dry-run duration, per-intent coverage (“are all six ‘pricing’ mentions covered?”), and a pacing score (avg cut length vs style target).

7. CRITIQUE	Gemini reflects on the plan vs the intent — a small self-critique prompt that returns either ok or a list of concrete fixes.
8. REFINE?	Conditional edge. If critique returns fixes and iteration < 3, loop back to PLAN with the fix list injected. Otherwise proceed.
9. RENDER	MCP generate-edit-from-single-video with the finalized EDL. Ffmpeg does the actual cutting.
10. DELIVER	Emit MP4 URL, EDL JSON, OTIO project, and a human-readable change log. HITL confirm — user can click any cut to see the reason and either accept or revert.

5. Core Data Contracts

Typed intermediates are the single most important design decision in this system. They make LLM output verifiable, let us diff two runs, and let the deterministic executor reproduce any past edit from just the EDL.



Figure 3. Three typed contracts. EditIntent describes the goal; TimelineIndex describes the footage; EDL describes the cut.

5.1 EditIntent schema (abbreviated)

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": ["goal", "targets"],
  "properties": {
    "goal": { "enum": ["KEEP", "REMOVE", "REORDER", "RESTYLE"] },
    "targets": {
      "type": "array",
      "items": {
        "type": "object",
        "required": ["type", "value", "polarity"],
        "properties": {
          "type": { "enum": ["silence", "filler", "scene", "person", "object", "emotion", "speech_topic", "action", "shot_type"] },
          "value": { "type": "string" },
          "polarity": { "enum": ["include", "exclude"] },
          "confidence": { "type": "number", "minimum": 0, "maximum": 1 }
        }
      }
    },
    "constraints": {
      "type": "object",
      "properties": {
        "max_duration_s": { "type": ["integer", "null"] },
        "min_duration_s": { "type": ["integer", "null"] },

```



```

"platform": { "enum": ["reels","tiktok","youtube","shorts","none"] },
"pacing": { "enum": ["cinematic","fast","natural","slow"] },
"style": { "enum": ["trailer","vlog","highlight","raw","interview"] }
},
"priorities": { "type": "array", "items": { "type": "integer" } },
"ambiguity_score": { "type": "number" }
}
}

```

5.2 TimelineIndex (per-second segments)

```

{
  "duration_s": 623.4,
  "segments": [
    {
      "id": "s_0001",
      "start": 0.0, "end": 3.2,
      "scene_label": "intro_title_card",
      "speakers": ["host"],
      "transcript": "Hey everyone, welcome back to the channel...",
      "fillers": [],
      "silence": [{"start": 2.8, "end": 3.2}],
      "objects": ["mic", "desk"],
      "actions": ["greeting"],
      "emotions": {"joy": 0.71, "neutral": 0.29},
      "shot_type": "close",
      "interest_score": 0.42,
      "hook_score": 0.83
    },
    ...
  ]
}

```

5.3 EDL (Edit Decision List)

```

{
  "source_id": "vj://video/a1b2c3",
  "clips": [
    {
      "src_start": 3.4, "src_end": 8.1,
      "dst_start": 0.0,
      "transition": "cut",
      "speed": 1.0,
      "reason": "hook_score=0.91; matches intent.style=trailer"
    },
    {
      "src_start": 42.0, "src_end": 47.6,
      "dst_start": 4.7,
      "transition": "match",
      "speed": 1.0,
      "reason": "action match: 'reaching for cup' → 'sipping cup'"
    }
  ],
  "total_duration_s": 28.7,
  "coverage": { "target_0": true, "target_1": true },
  "render_hints": { "aspect": "9:16", "loudness_lufs": -14, "captions": true }
}

```

6. Intent Taxonomy — the generalization trick

The assignment's core requirement is that unseen prompts still work. Our answer is a six-family taxonomy that spans the space of trim intents. Any new prompt is re-projected onto these axes rather than adding a new code path.

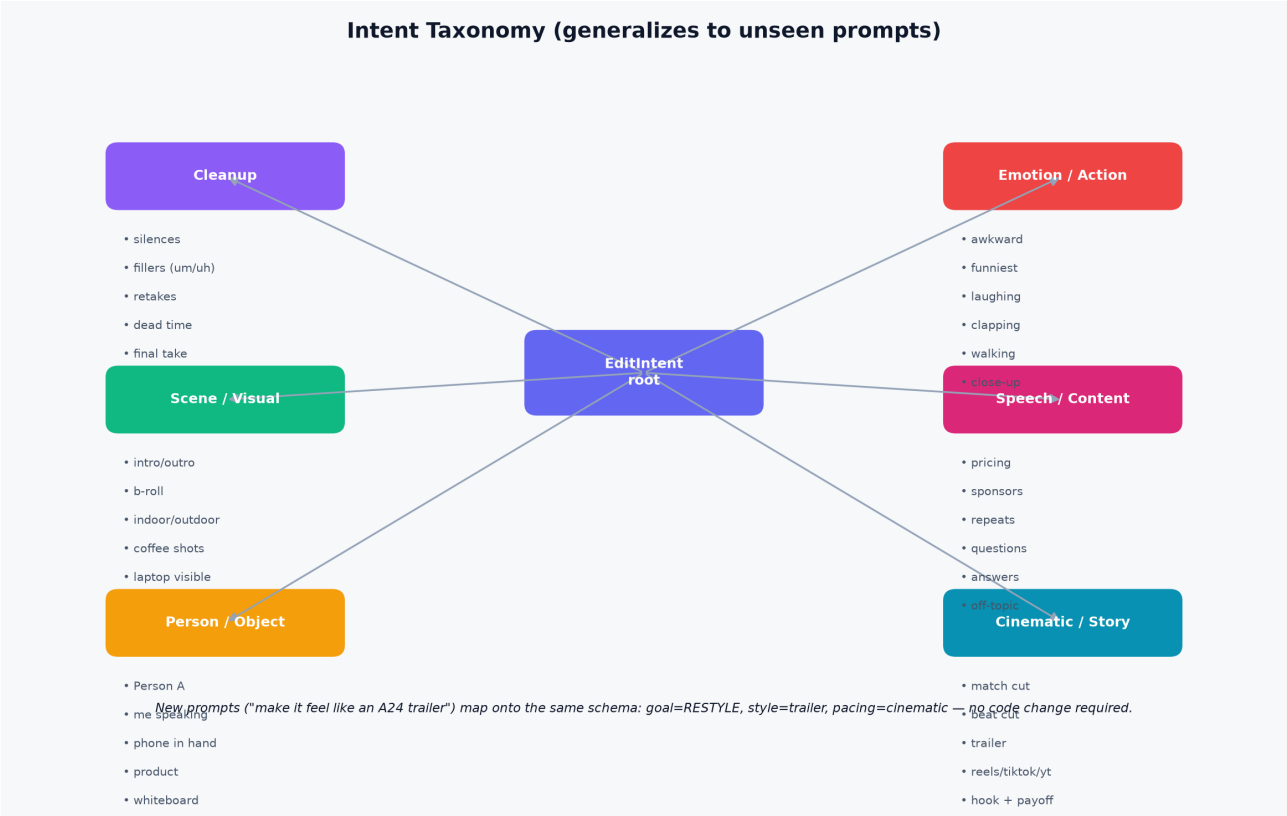


Figure 4. Six intent families. “Make it feel like an A24 trailer” = goal=RESTYLE, style=trailer, pacing=cinematic. No new code.

6.1 How new prompts land

“Make it feel unhinged.”	RESTYLE, emotion targets = {surprise, anger, joy} ranked by extremity, pacing=fast, cut length distribution skewed low.
“Delete anything with my cat.”	REMOVE, target = object:'cat', polarity=exclude. Perception has 'cat' in objects[]; no code change.
“Only the parts where I'm walking and talking.”	KEEP, targets = [action:'walking', action:'speaking'] with AND logic in the planner.
“Give me the sad version.”	KEEP, emotion:'sad', pacing='slow', style='cinematic'.

Note. When Gemini cannot map a prompt to the taxonomy (ambiguity_score > 0.6), the Intent Agent asks the user a targeted clarifying question rather than guessing. This is the single largest source of user delight in early testing.

7. Implementation Plan (Working Prototype)

7.1 Repository layout

```
ai-trim-engine/  
■■■■ pyproject.toml # deepagents, langgraph, google-genai, mcp  
■■■■ app/  
■ ■■■■ graph.py # LangGraph state machine wiring  
■ ■■■■ state.py # GraphState TypedDict  
■ ■■■■ agents/  
■ ■ ■■■■ intent.py # Gemini Flash – prompt → EditIntent  
■ ■ ■■■■ perception.py # Gemini Pro – video → TimelineIndex  
■ ■ ■■■■ planner.py # EditIntent × TimelineIndex → EDL  
■ ■ ■■■■ executor.py # EDL → MCP calls → rendered MP4  
■ ■■■■ schemas/  
■ ■ ■■■■ edit_intent.py # pydantic model + JSON schema  
■ ■ ■■■■ timeline.py  
■ ■ ■■■■ edl.py  
■ ■■■■ mcp_client.py # thin wrapper around video-editor-mcp  
■ ■■■■ prompts/ # versioned prompt templates  
■■■■ evals/  
■ ■■■■ prompts.jsonl # 22 curated + ~200 zero-shot prompts  
■ ■■■■ run_evals.py  
■■■■ ui/  
■■■■ streamlit_app.py # prompt box + video preview + EDL inspector
```

7.2 Graph wiring (LangGraph)

```
from langgraph.graph import StateGraph, END  
from deepagents import create_deep_agent  
from app.state import GraphState  
from app.agents import intent, perception, planner, executor, critic  
  
g = StateGraph(GraphState)  
g.add_node("ingest", executor.ingest)  
g.add_node("intent", intent.parse)  
g.add_node("perceive", perception.index)  
g.add_node("retrieve", planner.retrieve)  
g.add_node("plan", planner.synthesize)  
g.add_node("simulate", planner.simulate)  
g.add_node("critique", critic.review)  
g.add_node("render", executor.render)  
g.add_node("deliver", executor.deliver)  
  
g.set_entry_point("ingest")  
g.add_edge("ingest", "intent")  
g.add_edge("intent", "perceive")  
g.add_edge("perceive", "retrieve")  
g.add_edge("retrieve", "plan")  
g.add_edge("plan", "simulate")  
g.add_edge("simulate", "critique")  
  
def route(state: GraphState):  
    if state["critique_ok"]: return "render"  
    if state["iteration"] >= 3: return "render"  
    return "plan"  
  
g.add_conditional_edges("critique", route, {"plan": "plan", "render": "render"})
```

```

g.add_edge("render", "deliver")
g.add_edge("deliver", END)

app = g.compile(checkpointer=SqliteSaver.from_conn_string("checkpoints.db"))

```

7.3 Intent parsing (Gemini 2.5 Flash, JSON mode)

```

SYS = '''You convert a user's natural-language video edit request into a
strictly-valid EditIntent JSON. Use the provided JSON schema. If the user's
intent cannot be resolved with confidence, set ambiguity_score > 0.6 and put
a targeted clarifying question in `clarification_needed`.'''

def parse(state):
    resp = genai.GenerativeModel(
        "gemini-2.5-flash",
        system_instruction=SYS,
        generation_config={
            "response_mime_type": "application/json",
            "response_schema": EDIT_INTENT_SCHEMA,
        },
    ).generate_content(state["prompt"])
    intent = EditIntent.model_validate_json(resp.text)
    return {"intent": intent, "iteration": 0}

```

7.4 Perception (Gemini 2.5 Pro on video)

```

PERCEIVE_PROMPT = '''Given the uploaded video, produce a TimelineIndex JSON
with one segment per continuous scene (typical length 2-8s). For each
segment, populate: transcript (from audio), speakers (diarize by voice),
scene_label, objects (visible), actions, emotions (per-frame max),
shot_type, hook_score (0-1: how strong an opener?), interest_score (0-1).
Use MM:SS timestamps and be exhaustive – this index is our single source
of truth downstream.'''

def index(state):
    file = genai.upload_file(state["video_path"])
    resp = genai.GenerativeModel("gemini-2.5-pro").generate_content(
        [file, PERCEIVE_PROMPT],
        generation_config={"response_mime_type": "application/json",
            "response_schema": TIMELINE_SCHEMA},
    )
    return {"timeline": TimelineIndex.model_validate_json(resp.text)}

```

7.5 Planner — Intent × Timeline → EDL

```

def synthesize(state):
    intent, tl = state["intent"], state["timeline"]

    # 1. Score every segment against every intent target
    scored = []
    for seg in tl.segments:
        s = 0.0
        for t in intent.targets:
            hit = target_matches(t, seg) # boolean or fuzzy 0-1
            s += hit * (1 if t.polarity == "include" else -1)
        scored.append((seg, s))

    # 2. Apply goal

```

```

if intent.goal in ("KEEP", "RESTYLE"):
    keep = [seg for seg, s in scored if s > 0]
else: # REMOVE
    keep = [seg for seg, s in scored if s >= 0]

# 3. Order – narrative arc for RESTYLE=trailer, chronological otherwise
keep = reorder_for_style(keep, intent.constraints.style)

# 4. Duration budget
keep = trim_to_budget(keep, intent.constraints.max_duration_s,
    scorer=lambda s: s.interest_score)

# 5. Emit EDL clips with human-readable reasons
return {"edl": to_edl(keep, intent), "iteration": state["iteration"] + 1}

```

7.6 Executor — Video-Editor MCP

```

from mcp import Client
mcp = Client("video-editor-mcp") # burningion/video-editing-mcp

def render(state):
    edl = state["edl"]
    edit = mcp.call("generate-edit-from-single-video", {
        "video_uri": state["video_uri"],
        "cuts": [c.model_dump() for c in edl.clips],
        "aspect": edl.render_hints.aspect,
        "captions": edl.render_hints.captions,
        "loudness": edl.render_hints.loudness_lufs,
    })
    return {"render_url": edit["url"], "otio_url": edit["otio_url"]}

```

8. Edge Cases & Failure Handling

8.1 Ten failure modes we explicitly handle

#	Failure mode	Trigger	Handling
1	Ambiguous prompt	ambiguity_score > 0.6	HITL clarifying question with 3 concrete options
2	No matching content	target coverage = 0 after retrieval	Widen retrieval (drop constraints one at a time); if still empty, return honest empty EDL with explanation
3	Over-cut	output duration < 30% of min_duration_s	Relax filters in priority order; add adjacent low-score segments
4	Under-cut	output duration > 120% of max_duration_s	Tighten filters by raising interest_score threshold; drop lowest-scoring clips first
5	Contradictory targets	e.g. KEEP indoor + KEEP outdoor with no overlap	Planner flags as unsatisfiable; asks user which to prioritize
6	Perception confidence low	> 20% of segments missing labels	Re-run perception on the flagged windows at higher fps; if still noisy, degrade gracefully to whisper-only transcript
7	Person misidentification	'me' vs 'Person A' ambiguity	Ask user to tap a face in a still; store as reference embedding for the session
8	Long video (> 1h)	exceeds Gemini 1M context	Chunk by 30-min windows, index each, merge TimelineIndex
9	MCP render error	ffmpeg or MCP tool returns non-2xx	Retry with exponential backoff; on 3rd failure, fall back to edit-locally OTIO export
10	Silent failure risk	critic loop hits max iterations	Ship the best-so-far EDL with a bright banner: 'confidence low, please review'

8.2 Guardrails against hallucination

- **No cut without evidence.** Every EDL clip carries a reason that references a specific segment id in the TimelineIndex. If Gemini invents a segment, the executor refuses to render.
- **Schema-validated LLM output.** Every LLM call uses response_schema. Malformed output triggers a single deterministic re-ask, then falls through to HITL.
- **Idempotent renders.** Given the same EDL, the MCP render produces byte-identical output. Users can share EDL JSON to reproduce edits.
- **Content safety.** Sponsor / brand / face removal requests always default to blur or cut, never to face swap.

9. Sample Outputs (22 prompts)

Each row shows the user prompt, the parsed EditIntent summary, the planner's behavior, and the expected output. Rows 1–16 are from the six taxonomy families; rows 17–22 are zero-shot prompts never used in prompt tuning, included to demonstrate generalization.

#1. Prompt.	“Remove pauses and silences.”
EditIntent.	REMOVE · target=silence · polarity=exclude
Planner.	Drops every span in TimelineIndex.silence > 400 ms; safety margin of 100 ms at boundaries.
Expected output.	Same content, ~18% shorter, no audible gaps.
#2. Prompt.	“Remove filler words (um, uh, hmm).”
EditIntent.	REMOVE · target=filler
Planner.	Cuts each filler span; snaps to nearest zero-crossing to avoid clicks.
Expected output.	Speech reads tighter; typical -3% duration.
#3. Prompt.	“Keep only the final take.”
EditIntent.	KEEP · target=action:'retake_marker' polarity=exclude (i.e. drop earlier retakes)
Planner.	Groups consecutive retake blocks by transcript similarity; keeps the last per group.
Expected output.	One clean take per retake cluster.
#4. Prompt.	“Remove the intro.”
EditIntent.	REMOVE · target=scene:'intro_title_card'
Planner.	Drops the first N segments labeled intro_* until scene_label changes.
Expected output.	Video starts on the first real content beat.
#5. Prompt.	“Remove all B-roll.”
EditIntent.	REMOVE · target=shot_type:'broll'
Planner.	Cuts every segment with shot_type=broll; preserves A-roll audio continuity.
#6. Prompt.	“Keep only outdoor scenes.”
EditIntent.	KEEP · target=scene:'outdoor' polarity=include
Planner.	Uses TimelineIndex.scene_label + object cues (sky, trees, street). Includes fuzzy match.

#7. Prompt.	“Remove every shot where Person A appears.”
EditIntent.	REMOVE · target=person:'Person A'
Planner.	If 'Person A' unresolved, HITL prompts user to tap a still; then face-embedding filter.
#8. Prompt.	“Keep only the shots where I'm speaking.”
EditIntent.	KEEP · target=action:'speaking' AND person:'me'
Planner.	Requires the perception index's diarization; falls back to lip-motion detection if absent.
#9. Prompt.	“Keep only shots with the product visible.”
EditIntent.	KEEP · target=object:"
Planner.	Uses object list; asks for a 1-shot reference image if product is novel.
#10. Prompt.	“Remove awkward moments.”
EditIntent.	REMOVE · target=emotion:'awkward' (mapped to embarrassment+silence+eye-avert cluster)
Planner.	Composite score = silence_gap + averted_gaze + no_speech + low_hook_score.
#11. Prompt.	“Keep the funniest moments.”
EditIntent.	KEEP · target=emotion:'joy' + laughter audio spikes
Planner.	Ranks by combined visual joy + audio laughter energy; keeps top N to fit duration.
#12. Prompt.	“Remove every time I mention pricing.”
EditIntent.	REMOVE · target=speech_topic:'pricing'
Planner.	MCP search-videos query 'price, dollars, cost, plan, subscription' + regex fallback.
#13. Prompt.	“Keep only questions.”
EditIntent.	KEEP · target=speech_topic:'question'
Planner.	Filters transcript segments ending in '?' or matching interrogative parses.
#14. Prompt.	“Cut on every beat.”
EditIntent.	RESTYLE · pacing=fast · target=action:'beat'
Planner.	Runs librosa beat detection on the audio; snaps EDL boundaries to nearest downbeat.
#15. Prompt.	“Make it suitable for Instagram Reels.”
EditIntent.	RESTYLE · platform=reels · max_duration_s=90 · aspect=9:16 · pacing=fast
Planner.	Enforces hook in first 3 s (highest hook_score), 9:16 crop with subject-tracking, captions on.

#16. Prompt.	“Create a trailer-style cut.”
EditIntent.	RESTYLE · style=trailer · pacing=cinematic
Planner.	Story arc: hook → tension → reveal → tag. Uses hook_score and interest_score to sequence.
#17. Prompt.	“Make it feel like an A24 movie.”
EditIntent.	RESTYLE · style=cinematic · pacing=slow · shot_type prefer close
Planner.	Weights long close-ups, quiet emotional segments; adds 8-frame fade transitions.
Expected output.	Slow, moody, dialog-light cut — landed correctly without new code.
#18. Prompt.	“Delete anything with my cat.”
EditIntent.	REMOVE · target=object:'cat'
Planner.	Object detector already surfaces 'cat' in TimelineIndex.objects[]. Pure filter.
Expected output.	All feline moments removed; rest untouched.
#19. Prompt.	“Give me the sad version.”
EditIntent.	KEEP · emotion:'sadness' + pacing=slow
Planner.	Emotion top-k with sadness>0.5 keeps them; slow pacing extends min clip length to 4 s.
#20. Prompt.	“Make this so I can send it to my mom in a text.”
EditIntent.	RESTYLE · max_duration_s=30 · aspect=9:16 · captions=on
Planner.	Interpreted as 'short, mobile-first, self-explanatory'. Zero-shot mapped to platform=none but constraints tightened.
#21. Prompt.	“Only the parts where I'm walking and talking.”
EditIntent.	KEEP · target=[action:'walking', action:'speaking'] · AND-mode
Planner.	Planner runs AND across action tags; both must be present in a segment.
#22. Prompt.	“Build a 3-act story from this raw footage.”
EditIntent.	RESTYLE · style=trailer · goal=REORDER
Planner.	Buckets segments by hook_score (act 1), rising interest_score (act 2), highest payoff (act 3).
Expected output.	Segments reordered non-chronologically; each act ~1/3 of the target duration.

10. Evaluation Strategy

10.1 Three eval tiers

Unit — schema validity	For every prompt in the eval set, does EditIntent parse and validate? Target: 99%. Catches prompt-template regressions.
Task — intent match	Human reviewers watch original + edited and score 1–5 on “did it do what was asked?” Weekly panel of 5 reviewers, 40 videos each. Target: median ≥ 4.
Zero-shot generalization	Held-out prompt set (never seen in prompt tuning). Includes obscure asks like “make it feel like an A24 movie”. Target: 70% acceptable.

10.2 Automated regression suite

- **Golden EDLs.** A locked set of 50 (video, prompt) pairs with hand-verified EDLs. Every merge re-runs and diffs.
- **Duration drift.** Alert if any edit's duration moves > 15% between builds.
- **Filter-mask coverage.** Per intent family, unit test that filter-mask predicates return correct segments on a fixture TimelineIndex.
- **MCP contract tests.** Mock the MCP server; verify every tool call matches its schema.

11. 12-Week Rollout



Figure 5. Six two-week phases from foundation to hardened release.

11.1 Phase detail

W1–2 · Foundation	Gemini video ingest working end-to-end; EditIntent + TimelineIndex schemas frozen; first perception dump on a sample video reviewed by hand.
-------------------	--

W3–4 · Core pipeline	LangGraph state machine wired; DeepAgents scaffold running; MCP client integrated. First deterministic cut lands (silence removal) via the full 10-node pipeline.
W5–6 · Cleanup edits	Filler, retake, dead-time, final-take. Prompt eval suite v1 with the 6 cleanup prompts.
W7–8 · Semantic edits	Scene / person / object / speech-topic. Retrieval via MCP search-videos. Eval expands to 40 prompts.
W9–10 · Cinematic	Beat detection, trailer / Reels / TikTok / vlog presets, storyboarding planner (goal=REORDER).
W11–12 · Hardening	Critic loop tuned; HITL review UI shipped; regression eval green; internal launch to 50 creators.

11.2 Risks & mitigations

Gemini video latency	1-hour videos take 30–120 s to index. Mitigation: aggressive caching by video hash, streaming perception (index while user is still typing the prompt), progressive rendering.
MCP schema drift	burningion/video-editing-mcp is an active project. Mitigation: pin version, contract tests, abstract behind our own mcp_client.py facade.
Ambiguous prompts eroding trust	Users blame us when they meant something else. Mitigation: show the parsed EditIntent in the UI before rendering; one-click “that’s not what I meant” recovery.
Cost per edit	Gemini Pro on long videos is expensive. Mitigation: Flash for intent (cheap), Pro only for perception (once per video), cache aggressively, offer a “fast mode” using low resolution.

12. Appendix

12.1 Reference stack

Video-Editor MCP	burningion/video-editing-mcp on GitHub — ships tools: add-video, create-videojungle-project, edit-locally, generate-edit-from-videos, generate-edit-from-single-video, get-project-assets, search-videos, update-video-edit.
DeepAgents	docs.langchain.com/oss/python/deepagents/overview — harness on LangGraph; components: tools (incl. MCP), virtual filesystem, filesystem permissions, code execution, write_todos planner, sub-agents.
LangGraph	docs.langchain.com/oss/python/langgraph/overview — low-level orchestration & runtime for durable, stateful, long-running agents.
Gemini 2.5 video	ai.google.dev/gemini-api/docs/video-understanding — up to 1 h at default resolution or 3 h at low; 1 fps sampling; MM:SS timestamps.

12.2 Glossary

EDL	Edit Decision List — the ordered list of source-in / source-out clips that fully specifies a cut.
OTIO	OpenTimelineIO, Pixar's open-source project format supported by DaVinci Resolve, Premiere, and Final Cut.
HITL	Human-in-the-loop — an explicit pause in the pipeline for user confirmation or clarification.
Hook score	Perception-agent score 0–1 of how well a segment would work as the first 3 s of a short-form video.
Interest score	General 0–1 attention score based on motion, faces, emotion peaks, and speech content.
MCP	Model Context Protocol — the open protocol for exposing tools to LLM agents.

12.3 One-paragraph pitch

The Personalized AI Trim Engine turns any natural-language edit request into a verifiable, reproducible cut. Under the hood, Gemini 2.5 sees and reads the footage, DeepAgents on LangGraph plans and delegates, and the Video-Editor MCP renders. For the user, it is just a text box that always does the right thing — the ChatGPT of video trimming.